САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа №1

Выполнил:

Дьячкова Елизавета

J3212

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Пройти три игры:

- https://flexboxfroggy.com/#ru
- https://cssgridgarden.com/#ru
- https://learngitbranching.js.org/?locale=ru_RU (основы работы с git, достаточно выполнить первые 4 урока + все уроки по удалённым репозиториям)

Собрать отчёт по шаблону (https://docs.google.com/document/d/1zgXRYAh9iO21NDcp5Cua3mOiNRWbAVZJBfgGiqGsMsU/edit?usp=sharing). Экспортировать в pdf и открыть Pull Request в этот репозиторий.

Ход работы

В лабораторной работе мной были последовательно пройдены три обучающих игры, которые позволили изучить на практике закрепить теоретический материал.

1. Прохождение игры Flexbox Froggy

В этой игре я перемещала лягушек на кувшинки, используя различные свойства CSS Flexbox. Каждый уровень знакомил с новым свойством или значением.Снизу я привожу градацию уровней и полученные знания

№	Задание	Использованное CSS-свойство, пример	Краткое описание принципа работы
1 - 4	переместить лягушку	justify-content: flex-end;	Выравнивание элементов по главной оси в конце контейнера. **flex-start**: элементы выравниваются по левой стороне контейнера. **flex-end**: элементы выравниваются по правой стороне контейнера.

			center: элементы выравниваются по центру контейнера. **space-between**: элементы отображаются с одинаковыми отступами между ними. **space-around**: элементы отображаются с одинаковыми отступами вокруг них.
5 - 7	лягушке добраться до n части пруда	align-items: flex-end;	**align-items выравнивать элементы вертикально** **flex-start**: элементы выравниваются по левой стороне контейнера. **flex-end**: элементы выравниваются по правой стороне контейнера. **center**: элементы выравниваются по центру контейнера. **space-between**: элементы отображаются с одинаковыми отступами между ними. **space-around**: элементы отображаются с

			одинаковыми отступами вокруг них.
8-13	построить лягушек в том же порядке что и лилии	flex-direction: row-reverse;	**flex-direction -** это свойство CSS задает направление, в котором будут расположены элементы в контейнере row: элементы размещаются по направлению текста. **row-reverse:** элементы отображаются в обратном порядке к направлению текста. column: элементы располагаются сверху вниз. **column-reverse:** элементы располагаются снизу вверх.

14-15	задать положение через свойство	.yellow {
		order: 1;
		}
			order - это свойство для размещения

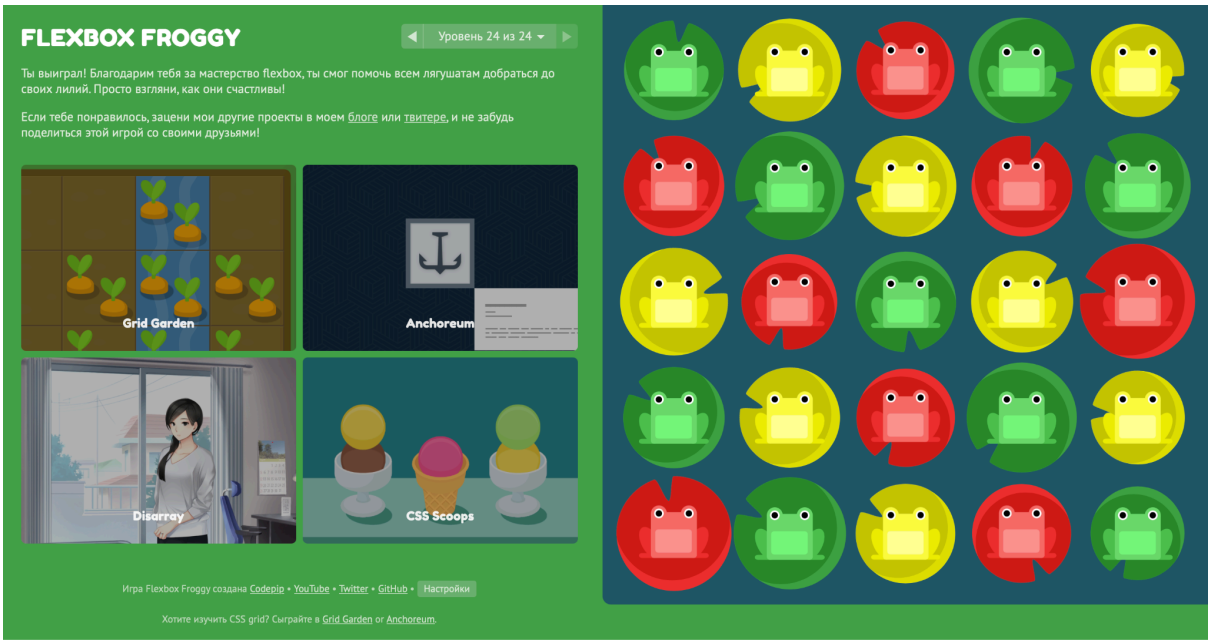
16-17	лягушке добраться до n части пруда	.yellow {
		  align-self: flex-end;
		}
			align-self - это свойство принимает те же значения, что и align-items.

18-19	Помоги этой армии лягушат выстроиться в три колонки	#pond {
		  display: flex;
		  flex-direction: column-reverse;
		  flex-wrap: wrap;
		}
			flex-wrap - это свойство для отступов, которое принимает следующие значения:
			nowrap: размеры элементов устанавливаются автоматически, чтобы они поместились в один ряд.
			wrap: элементы автоматически переносятся на новую строку.
			wrap-reverse: элементы автоматически переносятся на новую строку, но строки расположены в обратном порядке.

20-23	Помощь лягушке в размещение	Повторение комбинаций свойстав в css файлах	**-**

24	Финальный уровень на комбинацию всех свойств.	Например: flex-flow: column-reverse wrap-reverse; justify-content: center; align-content: space-between;	Комбинирование направления, переноса и выравнивания для достижения нужного расположения.

Вывод по разделу: В процессе игры были изучены и отработаны основные свойства Flexbox: justify-content (выравнивание по главной оси), align-items (выравнивание по поперечной оси), flex-direction (направление главной оси), order (изменение порядка), flex-wrap (перенос элементов) и align-content (выравнивание многострочного контейнера). Эти инструменты необходимы для создания css файлов.



2. Прохождение игры [Grid Garden]

Эта игра обучала управлению двухмерными сетками с помощью CSS Grid. Необходимо было поливать морковку, правильно задавая области для воды.

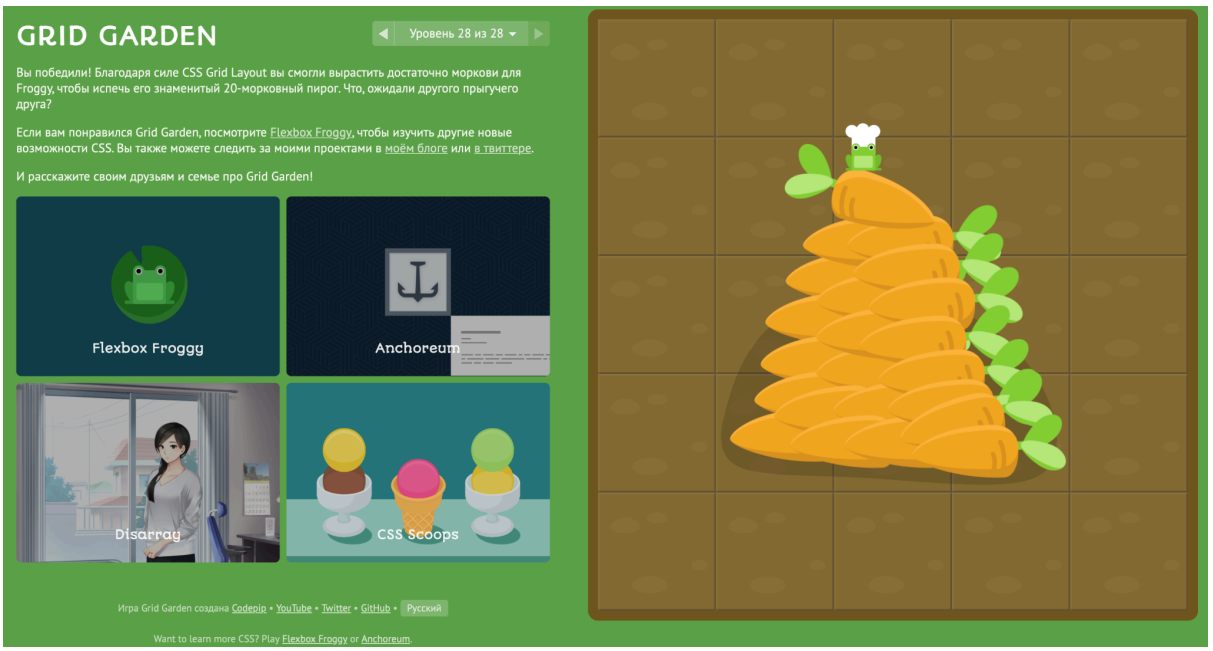
№	Задание	Использованное CSS-свойство и пример	Краткое описание принципа работы

1-4	Полить грядки, указывая их положение по вертикальным линиям сетки.	grid-column-start: 3;	Свойства grid-column-start и grid-column-end задают положение элемента по вертикали, указывая номера линий сетки, между которыми он должен находиться. Линии считаются с 1. Если конечная линия меньше начальной, элемент пойдёт справа налево.
5-7	Комбинировать начальную и конечную точки, использовать отрицательные индексы.	grid-column-end: -2;	Отрицательные значения отсчитывают линии с конца сетки. -1 - последняя линия, -2 - предпоследняя и т.д. Можно задавать только одно свойство, а другое оставлять по умолчанию (элемент будет занимать одну ячейку).
8-10	Использовать сокращённую запись и объединять строки и столбцы.	grid-column: 2 / 4;	Свойство grid-column - это сокращение для grid-column-start / grid-column-end. Аналогично grid-row работает для горизонтальных линий (строк).
11-13	Занимать несколько	grid-row: 5 / 7;	Элемент может занимать область, охватывающую

	ячеек по вертикали и горизонтали одновременно.		несколько строк и столбцов одновременно. Для этого нужно задать и grid-column, и grid-row.
14-17	Использовать ключевое слово span для указания размера.	grid-column: span 3;	Ключевое слово span указывает, сколько ячеек должен занимать элемент, вместо указания конечной линии. Например, span 3 означает "растянуться на 3 ячейки".
18-21	Задавать именованные области и использовать grid-area.	grid-area: 1 / 2 / 4 / 6;	Свойство grid-area — это супер-сокращение для grid-row-start / grid-column-start / grid-row-end / grid-column-end (порядок важен!).
22-24	Управлять порядком элементов с помощью order.	.water { order: 0; } .poison { order: 3; }	Свойство order работает так же, как во Flexbox: элементы с меньшим значением располагаются раньше в потоке сетки. По умолчанию у всех order: 0.

25-27	Использовать повторяющиеся шаблоны с repeat() и задавать доли fr.	grid-template-columns: 50% 50% 50% 50%;	Функция repeat() позволяет создать повторяющийся паттерн колонок или строк. Единица fr (fraction) задаёт гибкие размеры, пропорциональные свободному пространству.
28	Финальный уровень на комбинацию всех свойств.	grid-template-columns: 20% 80%; grid-template-rows: 1fr 50px;	Свойство grid-template объединяет grid-template-rows и grid-template-columns. Здесь создаётся сетка 2×4 с указанными размерами.

Вывод по разделу: Выполняя задания Grid Garden, были освоены ключевые концепции CSS Grid: определение линий и ячеек сетки (grid-column-start/end, grid-row-start/end), задание шаблона сетки (grid-template-columns, grid-template-rows), использование областей (grid-template-areas), а также выравнивание внутри ячеек.



3. Прохождение курса [Learn Git Branching]

Курс был пройден в два этапа: "Основы" (первые 4 урока) и "Удалённые репозитории" (все уроки).

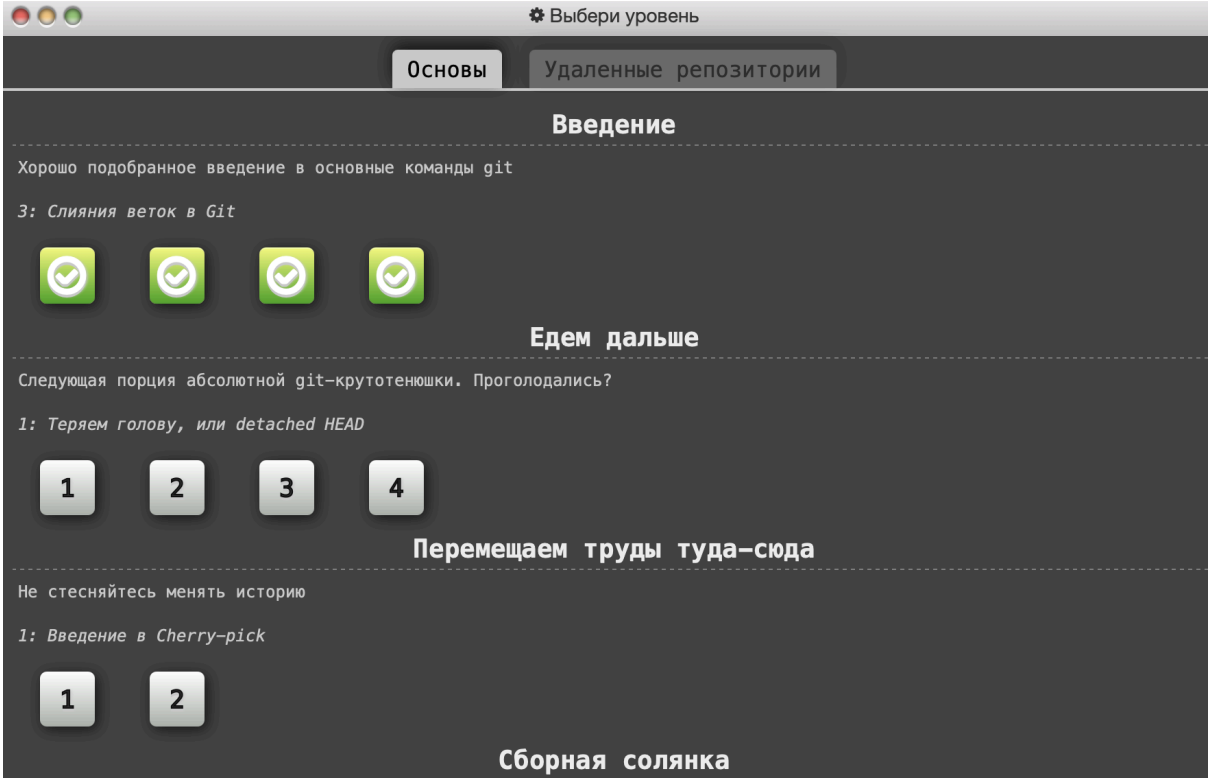
Этап 1: Основы работы с Git

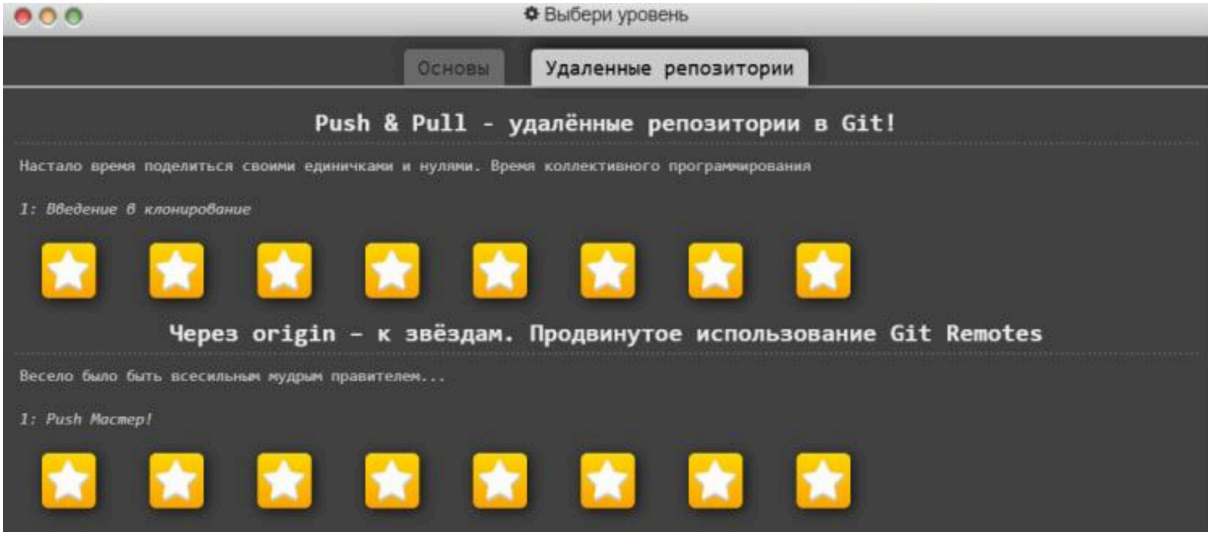
1. Уровень 1: Знакомство с Git Commit. Изучение команды git commit для создания "снимков" состояния проекта. Было выполнено два коммита подряд.
2. Уровень 2: Ветвление в Git. Освоение создания веток (git branch) и переключения между ними (git checkout). На практике было создано ответвление от основной линии разработки.
3. Уровень 3: Слияние веток (Merge). Изучение команды git merge для объединения изменений из разных веток. Был выполнен merge ветки bugFix в ветку main.
4. Уровень 4: Перемещение веток (Rebase). Изучение команды git rebase. В отличие от merge, rebase позволяет "переместить" изменения из одной ветки в другую, создавая более линейную историю коммитов.

Этап 2: Работа с удалёнными репозиториями

1. В этом блоке были изучены команды для синхронизации локального репозитория с удалённым (например, на GitHub).
2. git clone – создание локальной копии удалённого репозитория.
3. git fetch – получение изменений из удалённого репозитория без слияния с локальной веткой.
4. git pull – получение изменений и их автоматическое слияние с текущей локальной веткой (аналог git fetch + git merge).
5. git push – отправка локальных коммитов в удалённый репозиторий.
6. Также были рассмотрены сценарии работы с несколькими удалёнными ветками и разрешение конфликтов при слиянии.

Вывод по разделу: Прохождение курса позволило понять и закрепить на практике базовый цикл работы с Git: создание коммитов, работа с ветками, их слияние и перемещение. Важным результатом стало также освоение протокола взаимодействия с удалёнными репозиториями, что является необходимым навыком для любой командной разработки и для выполнения последующих лабораторных работ (например, создания Pull Request'ов).





Вывод

В процессе выполнения работы я разобралась с принципами работы Flexbox и CSS Grid. Формат работ был очень интересным, проще воспринимались именно через игровые, интерактивные задания.

Кроме того, я освоила базовые команды Git и получила чёткое понимание того, как функционирует система контроля версий: создание и управление ветками, их слияние, навигация по истории изменений и работа с удалёнными репозиториями.